



EchoPath — 项目技术文档

"Every path was once walked by someone before you."

EchoPath 是一款面向教育资源薄弱地区学生的职业导航与社交破冰平台。通过真实职业履历大数据分析，为学生展示"和自己起点相似的前辈"的成功轨迹，并借助 RAG 智能体生成高质量破冰邮件，帮助学生建立真实的导师连接。

目录

1. 产品概览与用户旅程
2. 系统架构总览
3. 技术栈与工具清单
4. 数据集需求
5. 核心模块详细设计
6. API 端点设计
7. 实施路线图
8. 优化方向与扩展策略
9. Hackathon 演示策略

1. 产品概览与用户旅程

1.1 核心问题

来自教育资源薄弱地区的学生面临三大障碍：

- 信息差：不知道"和自己起点类似的人"怎么走到成功的位置
- 人脉差：缺乏连接行业前辈的渠道和信心
- 行动差：即使找到目标人选，也不知道如何开口破冰

1.2 用户旅程流程

flowchart TD
 A["🎓 学生输入"] --> B["📍 地理 & 背景识别"]
 B --> C["🔍 多维匹配引擎"]
 C --> D["🗺️ 轨迹生成 - The Path"]
 C --> E["👤 榜样锚定 - The Match"]
 D --> F["📊 可视化职业路径展示"]
 E --> G["🧊 破冰行动 - The Action"]
 G --> H["✉️ RAG 生成个性化邮件"]
 H --> I["🤝 建立导师连接"]
 style A fill:#667eea,stroke:#333,color:#fff
 style F fill:#764ba2,stroke:#333,color:#fff
 style I fill:#f093fb,stroke:#333,color:#fff

1.3 用户输入字段

字段	类型	示例	用途
zip_code / fips_code	string	"06073"	定位教育资源水平
current_education	string	"社区大学" / "高中"	起点锚定
target_career	string	"大厂算法工程师"	目标职业映射
dream_description	text	"我想进大厂做算法..."	NLP 意图提取
school_name	string (optional)	"Modesto Junior College"	精准匹配校友

2. 系统架构总览

graph TB
 subgraph Frontend ["🖥️ Frontend - Next.js / Vite"]
 UI["用户输入界面"]
 PathViz["路径可视化"]
 MatchCard["榜样卡片"]
 EmailPreview["邮件预览 & 编辑"]
 end
 subgraph Backend ["⚙️ Backend - FastAPI / Flask"]
 API["REST API Gateway"]
 MatchEngine["多维匹配引擎"]
 PathBuilder["轨迹构建器"]
 EmailGen["邮件生成服务"]
 end
 subgraph Data ["📦 Data Layer"]
 PG["PostgreSQL - 结构化履历"]
 Redis["Redis - 缓存 & 会话"]
 Pinecone["Pinecone / Weaviate - 向量数据库"]
 end
 subgraph AI ["🧠 AI Layer"]
 Embed["Embedding Service"]
 RAG["RAG Engine - Rapidfire AI"]
 LLM["LLM - GPT-4 / Claude"]
 end
 subgraph External ["🌐 External Data"]
 Census["Census / NCES API"]
 GeoData["FIPS ↔ Zip 映射"]
 end
 UI --> API
 API --> MatchEngine
 API --> PathBuilder
 API --> EmailGen
 MatchEngine --> PG
 MatchEngine --> Pinecone
 PathBuilder --> PG
 EmailGen --> RAG
 RAG --> LLM
 LLM --> Embed
 Embed --> Pinecone
 External --> API

--> PG style Frontend fill:#1a1a2e,stroke:#667eea,color:#fff style Backend fill:#16213e,stroke:#0f3460,color:#fff style Data fill:#0f3460,stroke:#533483,color:#fff style AI fill:#533483,stroke:#e94560,color:#fff style External fill:#2c2c54,stroke:#aaa,color:#fff

3. 技术栈与工具清单

3.1 核心技术栈

层级	技术	理由
前端	Next.js 14 (App Router) + TypeScript	SSR SEO 友好 + 强类型
样式	Tailwind CSS + Framer Motion	快速原型 + 动画效果
可视化	D3.js / React Flow	职业路径图谱渲染
后端	Python FastAPI	高性能异步 + 原生 JSON 支持
数据库	PostgreSQL + JSONB	嵌套 JSON 原生支持 — <code>jobs[]</code> , <code>education[]</code> —
缓存	Redis	热门路径 & 匹配结果缓存
向量数据库	Pinecone / Weaviate / ChromaDB	语义相似度检索
LLM	OpenAI GPT-4o / Claude 3.5	邮件生成 & NLP 意图解析
RAG 框架	LangChain / LlamaIndex + Rapidfire AI	结构化数据增强生成
Embedding	OpenAI <code>text-embedding-3-small</code>	将职业轨迹向量化
部署	Vercel (前端) + Railway / Render (后端)	一键部署 `Hackathon 友好

3.2 开发工具

工具	用途
Jupyter Notebook	数据探索 & 特征工程原型
Pandas / NumPy	数据清洗 & 特征提取
scikit-learn	相似度计算 & 聚类分析
Pydantic	数据模型校验
Alembic	数据库迁移管理
Pytest	后端单元测试
Docker	本地开发环境一致性

3.3 第三方 API

API	用途	免费额度
U.S. Census API	FIPS Code → 地区经济/教育指标	免费
NCES API	学校层级数据（Title I 等）	免费
OpenAI API	Embedding + LLM 推理	按量付费
Rapidfire AI	RAG 增强内容生成	视赛事赞助
Google Geocoding	地理编码辅助	免费额度充足

4. 数据集需求

4.1 主数据集：职业履历数据（已有）

你们已有的数据集格式极为关键。以下是字段分析和利用策略：

核心字段提取策略

从原始 JSON 中提取的关键特征维度

```
PROFILE_FEATURES = {  
    # === 地理/起点特征 ===  
    "origin_fips": "从 jobs[] 中时间最早的 location_details.fips_code 提取",  
    "origin_msa": "最早期工作的 MSA (都市统计区)",  
    "origin_state": "最早期工作的 region",  
  
    # === 教育特征 ===  
    "education_school": "education[].school",  
    "education_degree": "education[].degree (学历层次)",  
    "education_field": "education[].field (专业方向)",  
  
    # === 职业轨迹特征 ===  
    "career_start_level": "jobs[] 按时间排序, 第一个 job 的 level",  
    "career_end_level": "当前 position.level",  
    "career_function": "position.function (所属职能)",  
    "total_tenure_months": "所有 job.duration 之和",  
    "company_count": "去重后的公司数量",  
    "industry_path": "按时间排列的 company.industry 序列",  
  
    # === 成功指标 ===  
    "current_company_size": "position.company.employee_count",  
    "current_company_type": "position.company.type (Public/Private)",  
    "level_progression": "从第一份工作 level 到当前 level 的跃升幅度",  
    "is_currently_employed": "employment_status == 'employed'",  
}
```

数据质量过滤规则

```
def is_valid_for_model(profile: dict) -> bool:
    """只有通过质量验证的履历才纳入模型。"""
    checks = [
        # 1. 必须有至少一段有效工作经历
        len([j for j in profile["jobs"] if j["title"] is not None]) >= 1,

        # 2. 教育信息不能全空
        len(profile.get("education", [])) > 0,

        # 3. 当前在职（确保是"成功案例"样本）
        profile.get("employment_status") == "employed",

        # 4. 职业 level 不能全为 null
        any(j.get("level") for j in profile["jobs"]),

        # 5. 至少有一个地理信息可用
        any(
            j.get("location_details", {}).get("fips_code")
            for j in profile["jobs"]
            if j.get("location_details")
        ),
    ]
    return all(checks)
```

4.2 补充数据集—需额外获取—

数据集	来源	用途	获取方式
FIPS → 经济指标	U.S. Census Bureau ACS	将 fips_code 映射为"教育资源薄弱程度"评分	Census API 免费
Title I 学校列表	NCES (National Center for Education Statistics)	标记来自低收入学区的学生/校友	NCES Data 免费下载
Zip Code ↔	HUD USPS	用户输入 Zip Code 转 FIPS	HUD API

数据集	来源	用途	获取方式
FIPS 映射表	Crosswalk		
大学排名/层级	Carnegie Classification / IPEDS	将 <code>education[].school</code> 映射为学校层级 (社区大学 / 州立 / 旗舰 / 常春藤)	IPEDS 免费下载
行业薪资中位数	BLS OEWS	辅助"成功"指标量化	BLS Data 免费
职业标准分类	O*NET / SOC Codes	将用户的模糊职业目标映射到标准分类	O*NET API

4.3 数据集规模建议

维度	Hackathon MVP	生产级别
职业履历条数	5,000 ~ 10,000	100,000+
覆盖地区 (FIPS)	Top 50 教育薄弱区	全美 3,000+ 县
行业覆盖	科技 + 金融 + 咨询	全行业
教育层级覆盖	社区大学 → 大厂	高中 → 任意终点

5. 核心模块详细设计

5.1 模块一：多维匹配引擎 (Match Engine)

核心算法思路

将每个用户的"起点状态"和每个前辈的"早期履历"分别编码为特征向量，计算加权余弦相似度。

```

import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.preprocessing import OneHotEncoder, MinMaxScaler

class MatchEngine:
    """多维匹配引擎：找到和当前学生起点最相似的前辈。"""

    # 特征权重 — 起始地理和教育背景权重最高
    WEIGHTS = {
        "geo_fips": 0.30, # 同一 FIPS Code → 最相关
        "geo_state": 0.10, # 同一州 → 次要加分
        "edu_tier": 0.25, # 校学层级相似度（社区大学 = 1, 州立 = 2, ...）
        "career_function": 0.15, # 目标职能匹配
        "economic_hardship": 0.20, # 起始地区经济困难指数
    }

    def encode_profile(self, profile: dict) -> np.ndarray:
        """将一条履历编码为特征向量。"""
        features = []

        # 1. 地理特征（One-Hot 或 Embedding）
        earliest_job = self._get_earliest_job(profile)
        fips = earliest_job.get("location_details", {}).get("fips_code", "unknown")
        features.append(self._geo_encode(fips))

        # 2. 教育层级（数值化）
        edu_tier = self._classify_school_tier(profile["education"])
        features.append(edu_tier)

        # 3. 职能方向（One-Hot）
        target_function = earliest_job.get("function", "unknown")
        features.append(self._function_encode(target_function))

        # 4. 经济困难指数（从 Census API 拉取）
        hardship_score = self._get_hardship_score(fips)
        features.append(hardship_score)

        return np.concatenate(features)

```



```

def find_matches(self, student_vector: np.ndarray,
                 alumni_vectors: np.ndarray,
                 top_k: int = 10) -> list:
    """返回 Top-K 最相似的前辈 ID 及相似度分数。"""
    # 加权余弦相似度
    weighted_student = student_vector * list(self.WEIGHTS.values())
    weighted_alumni = alumni_vectors * list(self.WEIGHTS.values())

    similarities = cosine_similarity(
        weighted_student.reshape(1, -1),
        weighted_alumni
    )[0]

    top_indices = np.argsort(similarities)[-top_k:][::-1]
    return [(idx, similarities[idx]) for idx in top_indices]

def _get_earliest_job(self, profile: dict) -> dict:
    """获取时间最早的有效工作经历。"""
    valid_jobs = [j for j in profile["jobs"] if j.get("title")]
    return sorted(valid_jobs, key=lambda j: j["started_at"])[0]

def _classify_school_tier(self, education: list) -> float:
    """对学校分层：0=未知，1=社区大学，2=州立，3=旗舰州立，4=私立名校，5=常春藤"""
    # 实际实现需查 IPEDS 数据库
    ...

def _geo_encode(self, fips: str) -> np.ndarray:
    """FIPS Code 地理编码。"""
    ...

def _function_encode(self, function: str) -> np.ndarray:
    """职能方向 One-Hot 编码。"""
    ...

def _get_hardship_score(self, fips: str) -> float:
    """根据 FIPS 获取经济困难指数 (0-1)。"""
    ...

```

学校层级分类逻辑

```
SCHOOL_TIER_MAP = {  
    # Tier 5: 顶尖私立 / 常春藤  
    "Harvard": 5, "Stanford": 5, "MIT": 5,  
    # Tier 4: 顶尖公立旗舰  
    "UC Berkeley": 4, "UCLA": 4, "University of Michigan": 4,  
    # Tier 3: 优质州立  
    "UC San Diego": 3, "University of Washington": 3,  
    # Tier 2: 一般州立  
    "San Diego State University": 2, "Cal State": 2,  
    # Tier 1: 社区大学  
    "community college": 1,  
    # 可通过 IPEDS Carnegie Classification 自动分类  
}
```

5.2 模块二：轨迹构建器 (Path Builder)

职业路径抽象模型

```

from dataclasses import dataclass
from typing import List

@dataclass
class CareerNode:
    """轨迹中的单个节点。"""
    stage: str          # "education" / "early_career" / "mid_career" / "senior"
    label: str           # "社区大学" / "州立大学 CS" / "中型科技公司"
    typical_duration: int # 平均停留月数
    count: int           # 有多少人经过这个节点

@dataclass
class CareerPath:
    """一条抽象的职业路径。"""
    nodes: List[CareerNode]
    total_people: int      # 经历此路径的总人数
    avg_years: float       # 平均总耗时 (年)
    success_rate: float    # 最终到达目标的比例

class PathBuilder:
    """从匹配到的前辈群体中，提取最常见的职业路径。"""

    LEVEL_ORDER = {
        "Intern": 0, "Staff": 1, "Senior Staff": 2,
        "Manager": 3, "Senior Manager": 4, "Director": 5,
        "VP": 6, "C-Suite": 7
    }

    def build_paths(self, matched_profiles: list,
                    target_function: str) -> List[CareerPath]:
        """
        输入：匹配到的前辈履历列表
        输出：Top-N 最常见的抽象路径
        """
        # Step 1: 将每个 profile 的 jobs[] 转换为有序节点序列
        sequences = []
        for profile in matched_profiles:
            seq = self._extract_sequence(profile, target_function)

```

```

        sequences.append(seq)

# Step 2: 用序列聚类（如 Smith-Waterman 对齐 or N-Gram）归纳路径
clustered = self._cluster_sequences(sequences)

# Step 3: 为每个聚类生成一条 CareerPath
paths = []
for cluster in clustered:
    path = self._summarize_cluster(cluster)
    paths.append(path)

return sorted(paths, key=lambda p: p.total_people, reverse=True)

def _extract_sequence(self, profile: dict, target_fn: str) -> list:
    """从一个 profile 中提取职业节点序列。"""
    nodes = []

    # 教育节点
    for edu in profile.get("education", []):
        nodes.append({
            "type": "education",
            "label": f"{edu.get('degree', '')} @ {edu.get('school', '')}",
            "tier": self._school_tier(edu["school"]),
        })

    # 工作节点（按时间排序）
    sorted_jobs = sorted(
        [j for j in profile["jobs"] if j.get("title")],
        key=lambda j: j["started_at"]
    )
    for job in sorted_jobs:
        nodes.append({
            "type": "career",
            "level": job.get("level", "unknown"),
            "function": job.get("function", "unknown"),
            "industry": job.get("company", {}).get("industry", "unknown"),
            "company_size": job.get("company", {}).get("employee_count", 0),
            "duration": job.get("duration", 0),
        })

```

```
        return nodes

def _cluster_sequences(self, sequences: list) -> list:
    """对职业序列进行聚类（简化版用 N-Gram）。"""
    ...

def _summarize_cluster(self, cluster: list) -> CareerPath:
    """将一个聚类总结为一条抽象 CareerPath。"""
    ...

def _school_tier(self, school_name: str) -> int:
    ...
```

5.3 模块三：RAG 邮件生成引擎 (Email Generator)

动态 Prompt 构建

```

class EmailGenerator:
    """基于 RAG 的个性化破冰邮件生成器。"""

    def build_prompt(self, student: dict, mentor: dict,
                     match_score: float) -> str:
        """
        动态拼接 prompt，不是静态模板，而是根据匹配模型的特征结果构建。
        """

        # 提取共同点
        common_origin = self._find_common_origin(student, mentor)
        career_highlights = self._extract_highlights(mentor)
        gap_analysis = self._analyze_gap(student, mentor)

        prompt = f"""你是一位专业的职业发展顾问，正在帮助一位学生写一封给潜在导师的破冰邮件。

## 学生信息
- 所在地区: {student['location']}
- FIPS Code: {student['fips_code']} (经济困难指数: {student['hardship_score']:.2f})
- 当前教育: {student['current_education']}
- 职业目标: {student['target_career']}
- 个人描述: "{student['dream_description']}"

## 导师信息 (匹配度: {match_score:.1%})
- 当前职位: {mentor['position']['title']} @ {mentor['position']['company']['name']}
- 职业级别: {mentor['position']['level']}
- 行业: {mentor['position']['company']['industry']}
- 教育背景: {self._format_education(mentor['education'])}
- 职业轨迹: {career_highlights}

## 共同连接点
{common_origin}

## 写作要求
1. 语气真诚、不卑不亢，展现学生的自驱力
2. 明确提到学生和导师的共同起点 (地理/学校背景)
3. 具体引用导师的某一段职业经历表示钦佩
4. 提出一个具体、低门槛的请求 (如 15 分钟电话)
5. 控制在 150-200 词以内

```


6. 用英文撰写

请直接输出邮件正文。"""

```
    return prompt

def _find_common_origin(self, student: dict, mentor: dict) -> str:
    """找到学生和导师的共同起点。"""
    commons = []
    mentor_fips_codes = set()
    for job in mentor["jobs"]:
        if job.get("location_details", {}).get("fips_code"):
            mentor_fips_codes.add(job["location_details"]["fips_code"])

    if student["fips_code"] in mentor_fips_codes:
        commons.append(f"- 同一地区 (FIPS: {student['fips_code']})")

    # 学校匹配
    student_schools = {student.get("current_school", "")}
    mentor_schools = {e["school"] for e in mentor.get("education", [])}
    overlap = student_schools & mentor_schools
    if overlap:
        commons.append(f"- 同一学校: {' '.join(overlap)}")

    return "\n".join(commons) if commons else "- 相似的起点背景和地理区域"

def _extract_highlights(self, mentor: dict) -> str:
    """提取导师的职业亮点。"""
    jobs = sorted(
        [j for j in mentor["jobs"] if j.get("title")],
        key=lambda j: j.get("started_at", ""),
    )
    highlights = []
    for job in jobs[-3:]: # 最近 3 份工作
        co = job.get("company", {}).get("name", "Unknown")
        title = job.get("title", "Unknown")
        highlights.append(f"{title} @ {co}")
    return " → ".join(highlights)

def _analyze_gap(self, student: dict, mentor: dict) -> dict:
```

```
"""分析学生和导师之间的差距，用于生成更有针对性的内容。"""
```

```
...
```

```
def _format_education(self, education: list) -> str:
```

```
    return "; ".join(
```

```
        f"{e.get('degree', 'N/A')} in {e.get('field', 'N/A')} @ {e.get('school', 'N/A')}
```

```
        for e in education
```

```
    )
```

5.4 模块四：地区困难度评分器 (Hardship Scorer)

```

class HardshipScorer:
    """根据 FIPS Code 计算 "教育资源薄弱度" 综合评分。"""

    def __init__(self, census_api_key: str):
        self.api_key = census_api_key
        self._cache = {} # FIPS → score 缓存

    INDICATORS = {
        "poverty_rate": 0.30, # 贫困率 (B17001)
        "no_bachelor_rate": 0.25, # 25+ 岁无本科学历比例 (B15003)
        "median_income": 0.20, # 家庭收入中位数 (B19013) – 反向
        "unemployment_rate": 0.15, # 失业率 (B23025)
        "title1_school_pct": 0.10, # Title I 学校占比
    }

    async def score(self, fips_code: str) -> float:
        """
        返回 0-1 的困难度分数, 1 = 最困难。
        """
        if fips_code in self._cache:
            return self._cache[fips_code]

        raw = await self._fetch_census_data(fips_code)
        normalized = self._normalize(raw)

        score = sum(
            normalized[indicator] * weight
            for indicator, weight in self.INDICATORS.items()
        )

        self._cache[fips_code] = score
        return score

    async def _fetch_census_data(self, fips_code: str) -> dict:
        """从 Census API 获取原始指标数据。"""
        # 使用 ACS 5-Year Estimates
        # GET https://api.census.gov/data/2022/acs/acs5
        # ?get=B17001_002E,B15003_001E,...
        # &for=county:{fips_code[2:]}

```

```
#     &in=state:{fips_code[:2]}
#     &key={self.api_key}
...

def _normalize(self, raw: dict) -> dict:
    """将原始数据归一化到 0-1。"""
    ...
```

6. API 端点设计

6.1 端点总览

方法	路径	描述
POST	/api/v1/student/profile	提交学生信息
GET	/api/v1/paths/generate	生成职业路径
GET	/api/v1/matches	获取匹配的榜样列表
GET	/api/v1/match/{id}	获取单个榜样详情（匿名化）
POST	/api/v1/email/generate	生成破冰邮件
GET	/api/v1/hardship/{fips}	查询地区困难度分数
GET	/api/v1/stats/overview	全局统计数据

6.2 关键端点示例

```
# FastAPI 路由示例
```

```
@app.post("/api/v1/student/profile")
```

```
async def create_student_profile(student: StudentInput):
```

```
    """
```

```
    接收学生输入，返回匹配结果 + 路径。
```

```
    """
```

```
    # 1. 解析 FIPS
```

```
    fips = await resolve_fips(student.zip_code or student.fips_code)
```

```
    # 2. 计算困难度
```

```
    hardship = await hardship_scorer.score(fips)
```

```
    # 3. NLP 解析职业目标
```

```
    parsed_goal = await parse_career_goal(student.dream_description)
```

```
    # 4. 构建学生向量
```

```
    student_vector = match_engine.encode_student({
        "fips_code": fips,
        "hardship_score": hardship,
        "current_education": student.current_education,
        "target_function": parsed_goal["function"],
        "target_level": parsed_goal["level"],
    })
```

```
    # 5. 多维匹配
```

```
    matches = match_engine.find_matches(student_vector, alumni_db, top_k=10)
```

```
    # 6. 生成路径
```

```
    paths = path_builder.build_paths(
        [alumni_db[m[0]] for m in matches],
        target_function=parsed_goal["function"]
    )
```

```
    return {
```

```
        "hardship_score": hardship,
```

```
        "parsed_goal": parsed_goal,
```

```
        "paths": paths[:3],          # Top 3 路径
```

```
        "matches": matches[:5],      # Top 5 榜样
```

```
}
```

```
@app.post("/api/v1/email/generate")
async def generate_email(request: EmailRequest):
    """
    为指定学生-导师配对生成破冰邮件。
    """
    student = await get_student(request.student_id)
    mentor = await get_mentor(request.mentor_id)
    match_score = request.match_score

    prompt = email_generator.build_prompt(student, mentor, match_score)

    # 调用 Rapidfire AI / LLM
    email_text = await rapidfire_ai.generate(
        prompt=prompt,
        context_documents=[mentor], # RAG 上下文
        temperature=0.7,
        max_tokens=500,
    )

    return {
        "email": email_text,
        "mentor_name_anonymized": anonymize(mentor["name"]),
        "match_score": match_score,
    }
```

7. 实施路线图

Phase 0: 数据准备 (Day 1 上午)

- ☐ 搭建 Python 数据处理脚本 '清洗现有 JSON 履历'
- ☐ 实现 `is_valid_for_model()` 数据质量过滤
- ☐ 从 Census API 拉取 FIPS → 经济指标映射
- ☐ 构建 IPEDS 学校层级查找表

- ☐ 将清洗后的数据导入 PostgreSQL (JSONB)

Phase 1: 核心引擎 (Day 1 下午)

- ☐ 实现 `MatchEngine` — 特征提取 + 加权余弦相似度
- ☐ 实现 `PathBuilder` — 序列提取 + 简化聚类
- ☐ 实现 `HardshipScorer` — Census API 集成
- ☐ 编写 Jupyter Notebook 验证匹配质量

Phase 2: API 层 (Day 1 晚上)

- ☐ FastAPI 项目搭建 + Pydantic 模型定义
- ☐ 实现 `/student/profile` 和 `/matches` 端点
- ☐ 实现 `/email/generate` + Rapidfire AI 集成
- ☐ 端到端测试：输入 → 匹配 → 路径 → 邮件

Phase 3: 前端 (Day 2 上午)

- ☐ Next.js 项目初始化 + Tailwind 配置
- ☐ 学生输入页面 (表单 + 地图选择器)
- ☐ 路径可视化组件 (D3.js / React Flow 桑基图)
- ☐ 榜样卡片组件 (匿名化展示)
- ☐ 邮件生成/预览/编辑组件

Phase 4: 打磨 & Demo (Day 2 下午)

- ☐ 端到端流程串通
 - ☐ 加入动画和微交互
 - ☐ 准备 Demo 数据 (预设场景)
 - ☐ 录制/准备演示 Pitch
-

8. 优化方向与扩展策略

8.1 算法优化

方向	当前方案	优化方案	效果
匹配算法	加权余弦相似度	Graph Neural Network (GNN) 在职业图谱上学习嵌入	更好地捕捉非线性关系
路径聚类	N-Gram 简单聚类	序列对齐 (Smith-Waterman) + DBSCAN	路径归纳更准确
意图解析	关键词匹配	Fine-tuned BERT 分类器 → <code>function</code> + <code>level</code>	模糊输入支持更好
Embedding	特征工程手动编码	职业轨迹 Sentence Transformer → 向量语义搜索	冷启动更友好

8.2 数据优化

方向	描述
数据增强	对 <code>location_details</code> 缺失的记录 ' 通过 <code>company.address</code> 反向填充 FIPS
时间衰减	更高权重给近 5 年内完成"起点→成功"转变的前辈∪更具参考性∩
置信度过滤	利用 <code>started_at_year_only</code> 字段 ' 对时间精度低的记录降权
去重	同一人多条记录合并∪通过 <code>education</code> + 早期 <code>jobs</code> 指纹匹配∩

8.3 产品优化

方向	描述
双语支持	考虑西语裔社区需求 ' 增加西班牙语界面

方向	描述
隐私合规	所有展示信息匿名化 '仅显示职位+公司规模级别（非具体公司名）'
导师验证	增加 opt-in 机制 '让前辈主动加入导师池'
进度追踪	学生可标记当前阶段 '系统动态更新推荐路径'
移动端优先	教育资源薄弱地区学生更依赖手机访问
社区功能	同一路径上的学生可以组队互助

8.4 性能优化

```
# 向量匹配预计算策略
OPTIMIZATION_STRATEGIES = {
    "precompute_embeddings": {
        "desc": "离线预计算所有校友的特征向量，存入向量数据库",
        "tool": "Pinecone / FAISS",
        "benefit": "匹配查询从 O(N) 降到 O(log N)",
    },
    "path_cache": {
        "desc": "高频路径组合缓存在 Redis 中",
        "ttl": "24h",
        "benefit": "相同起点的学生复用路径计算结果",
    },
    "batch_census": {
        "desc": "Census API 数据离线批量拉取并存表",
        "frequency": "每年更新",
        "benefit": "避免实时 API 调用延迟",
    },
}
```

9. Hackathon 演示策略

9.1 Demo 脚本

📍 场景：一个来自加州 Modesto (FIPS: 06099, 经济困难指数 0.72) 的高中生

1. [输入] 学生输入 Zip Code + "我想进大厂做市场营销"
2. [展示] 系统立刻标记 Modesto 为教育资源薄弱区 (hardship: 0.72)
3. [路径] "在数据库中, 有 87 位来自类似背景的前辈到达了 Marketing Manager 级别"
→ 最常见路径: 州立大学 → PR 公关公司 (积累 2年) → 大型广告集团 Manager
4. [匹配] "找到一位校友, 也来自 Modesto, 目前在 dentsu 任 Client Marketing Manager"
→ 展示匿名化卡片
5. [邮件] 点击 "Connect", 3 秒内生成个性化破冰邮件
→ 邮件中提到 Modesto 背景 + 对方在 dentsu 的经历

9.2 技术亮点 Pitch 要点

评委最关心的不是 UI 漂亮(虽然也重要) , 而是 :

1. 数据深度 : 告诉评委你们不是简单拿 JSON 展示 , 而是构建了一套完整的 ETL pipeline :
 - 嵌套 JSON → 特征提取 → 向量化 → 多维匹配
2. 算法公平性 : 强调你们的匹配模型加大了"起始地理位置"和"经济困难指数"的权重 , 这确保推荐结果是"接地气"的 , 而非推荐常春藤校友
3. **RAG** 不是玩具 : 你们的 RAG prompt 是基于结构化数据分析的动态构建 , 不是固定模板
4. 社会影响力 : 这不只是技术 demo , 而是真实解决教育公平问题

9.3 文件结构参考

```
EchoPath/
├── README.md
├── PROJECT_DOC.md          ← 本文档
├──
├── backend/
│   ├── main.py             # FastAPI 入口
│   ├── config.py           # 环境变量 & 配置
│   ├── models/
│   │   ├── student.py      # Pydantic 学生模型
│   │   ├── mentor.py       # Pydantic 导师模型
│   │   └── path.py         # 路径数据模型
│   ├── engines/
│   │   ├── match_engine.py # 多维匹配引擎
│   │   ├── path_builder.py # 轨迹构建器
│   │   ├── email_generator.py # RAG 邮件生成
│   │   └── hardship_scorer.py # 困难度评分器
│   ├── services/
│   │   ├── census_service.py # Census API 封装
│   │   ├── embedding_service.py # Embedding 封装
│   │   └── llm_service.py    # LLM / Rapidfire 封装
│   ├── data/
│   │   ├── raw/             # 原始 JSON 数据
│   │   ├── processed/       # 清洗后数据
│   │   └── scripts/
│   │       ├── etl.py        # 数据清洗脚本
│   │       └── seed_db.py    # 数据库填充脚本
│   └── tests/
│       ├── test_match.py
│       ├── test_path.py
│       └── test_email.py
├──
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   │   ├── page.tsx     # 首页 (学生输入)
│   │   │   ├── paths/page.tsx # 路径展示页
│   │   │   ├── match/page.tsx # 榜样详情页
│   │   │   └── email/page.tsx # 邮件生成页
│   │   └── components/
```

```
| | | |─ PathVisualization.tsx
| | | |─ MentorCard.tsx
| | | |─ EmailPreview.tsx
| | | |─ HardshipBadge.tsx
| | | └─ lib/
| | |   └─ api.ts          # 后端 API 封装
| | └─ tailwind.config.ts
| └─ package.json
|
└─ notebooks/
  └─ 01_data_exploration.ipynb
  └─ 02_feature_engineering.ipynb
  └─ 03_match_quality.ipynb
|
└─ docker-compose.yml
└─ .env.example
└─ requirements.txt
```

附录：数据字段速查

关键字段与用途映射

原始字段路径	用途	模块
jobs[0].location_details.fips_code	起始地理锚定	MatchEngine
jobs[].level	职业阶梯建模	PathBuilder
jobs[].function	职能方向匹配	MatchEngine
jobs[].company.employee_count	公司规模分类	PathBuilder
jobs[].company.industry	行业路径绘制	PathBuilder
jobs[].duration	阶段停留时长	PathBuilder
jobs[].company_tenure	公司忠诚度指标	PathBuilder

原始字段路径	用途	模块
<code>jobs[].is_first_at_company</code>	标识首份工作	MatchEngine
<code>jobs[].is_last_at_company</code>	标识当前活跃	MatchEngine
<code>education[].school</code>	学校层级分类	MatchEngine
<code>education[].degree</code>	学历层级	MatchEngine
<code>education[].field</code>	专业方向匹配	MatchEngine
<code>position.level</code>	当前成就级别	成功指标
<code>position.company.type</code>	公司类型分类	成功指标
<code>employment_status</code>	在职验证	数据过滤
<code>location_details.msa</code>	都市区归类	HardshipScorer
<code>connections</code>	社交活跃度指标	匹配权重



最后提醒：本项目的核心竞争力不在于 UI 有多花哨，而在于 "数据深度 × 算法公平性 × 社会影响力" 的三角形。确保 Demo 时评委能感受到：这不只是调了几个 API，而是一套有数据支撑、有算法逻辑、有社会关怀的完整系统。